

2025/2026

UniRide

Technical Documentation

ISPW Course Project
Ingegneria del Software

Alessio Carta

INDEX

1. SOFTWARE REQUIREMENT SPECIFICATION	3
1.1. Introduction	3
1.2. User Stories	4
1.3. Functional Requirements	5
1.4. Use Cases	6
2. STORYBOARDS	7
3. DESIGN	8
3.1. Class Diagram — VOPC (Analysis)	8
3.2. Class Diagram — Design Level	12
3.3. Design Patterns	13
3.4. Activity Diagram	14
3.5. Sequence Diagram	15
3.6. State Diagram	16
4. TESTING	18
DELIVERABLE LINKS	18

I numeri di pagina sono riferiti all'esportazione PDF del documento; potrebbero variare leggermente aprendo il file in Word in base ai font disponibili sul sistema.

1. SOFTWARE REQUIREMENT SPECIFICATION

1.1. Introduction

1.1.1. Aim of the Document

Il presente documento ha lo scopo di presentare in modo oggettivo i requisiti fondamentali e specifici del sistema UniRide. Attraverso descrizioni testuali, user story, diagrammi dei casi d'uso e artefatti di design, vengono ripercorsi tutti gli aspetti interni ed esterni rilevanti del sistema, fungendo da riferimento autorevole per lo sviluppo, il testing e la valutazione.

1.1.2. Overview of the Defined System

Il sistema qui esposto è un'applicazione desktop che aiuta gli studenti universitari a organizzare passaggi in auto condivisi da e per il campus. L'obiettivo primario è ridurre i costi di viaggio, le emissioni di CO₂ e favorire la socializzazione fra studenti dello stesso ateneo.

Sono supportati due ruoli dal sistema:

- Guidaatore (Studente) — pubblica un nuovo passaggio specificando partenza, destinazione, data, posti disponibili e costo base stimato. Le richieste di prenotazione in arrivo vengono confermate o rifiutate esplicitamente dal guidaatore stesso; la corsa può inoltre essere segnata come completata o annullata in qualsiasi momento.
- Passeggero (Studente) — consulta i passaggi disponibili, filtrandoli per partenza e destinazione, e richiede un posto. Il posto viene riservato immediatamente per evitare overbooking, ma la richiesta resta in attesa finché il guidaatore non la conferma o la rifiuta. Il passeggero può annullare una richiesta o una prenotazione già confermata in qualsiasi momento, liberando automaticamente il posto.

Il sistema gestisce il ciclo di vita completo di un Ride (AVAILABLE, FULL, COMPLETED, CANCELLED, tramite il Pattern State) e di una Booking (REQUESTED, CONFIRMED, REJECTED, CANCELLED), notificando i passeggeri coinvolti ad ogni transizione rilevante tramite il Pattern Observer.

1.1.3. Hardware and Software Requirements

Il requisito hardware di base è un computer in grado di eseguire una Java Virtual Machine, oltre a un minimo spazio su disco per i file CSV quando è attiva la modalità di persistenza su file system.

I requisiti software sono i seguenti:

Requirement	Detail
JDK	Java 25 o superiore (OpenJDK o Oracle JDK)
Build Tool	Apache Maven 3.8 o superiore
GUI Toolkit	JavaFX 21.0.5 (risolto automaticamente da Maven)
Sistema Operativo	macOS, Windows 10+, o Linux (Ubuntu 20+)
RAM	Minimo 512 MB liberi
Spazio Disco	Circa 50 MB (JAR più eventuali file CSV)
Persistenza	In-Memory (default) o file system CSV, selezionabile in Config.PERSISTENCE_TYPE

1.1.4. Related Systems, Pros and Cons

BLABLACAR

BlaBlaCar è il principale marketplace europeo di carpooling, che connette milioni di guidatori e passeggeri per viaggi tra città. Tra i pro: una base utenti molto ampia, un sistema di pagamento integrato e profili guidatore verificati. Tra i contro: non è pensato per studenti o tratte universitarie, richiede una transazione economica per ogni prenotazione e non ha alcun concetto di campus universitario come punto di riferimento.

GOMORE

GoMore è una piattaforma scandinava di ride-sharing che supporta anche il noleggio auto tra privati. A favore ha offerte di passaggio flessibili e un'interfaccia mobile pulita e moderna. Tuttavia manca di funzionalità legate alla community accademica (nessun filtro per università, nessuna verifica studentesca), impone un modello ad abbonamento ai guidatori e non offre una modalità demo offline o in memoria, rendendola poco adatta come riferimento per un'applicazione desktop Java in stile BCE.

1.2. User Stories

1.2.1. US-1

US-1

Come Studente, voglio pubblicare l'offerta di un passaggio specificando partenza, destinazione, data, posti disponibili e costo totale, così che altri studenti possano trovarlo e richiedere un posto nella mia auto.

1.2.2. US-2

US-2

Come Studente, voglio cercare i passaggi disponibili filtrando per partenza e destinazione, così da trovare rapidamente un passaggio adatto senza scorrere offerte irrilevanti.

1.2.3. US-3

US-3

Come Studente, voglio richiedere un posto su un passaggio ed essere avvisato quando il guidatore lo conferma o lo rifiuta, così da sapere sempre lo stato reale della mia prenotazione.

1.2.4. US-4

US-4

Come Studente guidatore, voglio poter confermare o rifiutare le richieste di prenotazione ricevute, segnare una corsa come completata o annullarla anche con passeggeri già a bordo, così da avere pieno controllo sul mio passaggio.

1.3. Functional Requirements

1.3.1. FR-1

FR-1 — Autenticazione

Il sistema consente a uno studente di registrarsi con username, password e dati anagrafici, e di accedere con le proprie credenziali.

1.3.2. FR-2

FR-2 — Gestione Passaggio

Il sistema consente a un guidatore autenticato di pubblicare, completare o annullare un proprio passaggio.

1.3.3. FR-3

FR-3 — Prenotazione

Il sistema consente a un passeggero autenticato di cercare un passaggio disponibile e richiedere un posto, riservato immediatamente.

1.3.4. FR-4

FR-4 — Gestione delle Richieste

Il sistema consente al guidatore di confermare o rifiutare una richiesta di prenotazione, e a entrambe le parti di annullarla in qualunque momento.

Glossario:

Passaggio (Ride) = Un tragitto offerto da uno studente guidatore, con partenza, destinazione, data, orario, posti disponibili e costo base.

Richiesta di Prenotazione (Booking) = La richiesta di un passeggero di occupare un posto su un Passaggio: il posto viene riservato subito, ma resta in attesa di conferma del guidatore.

Ateneo più vicino = Il campus universitario, tra quelli noti al sistema, geograficamente più vicino alla posizione dichiarata dallo studente; suggerito come destinazione predefinita in Offri/Cerca Passaggio.

Stato del Passaggio = La fase del ciclo di vita di un Passaggio (AVAILABLE, FULL, COMPLETED, CANCELLED), governata dal Pattern State.

Stato della Richiesta = La fase del ciclo di vita di una Richiesta di Prenotazione (REQUESTED, CONFIRMED, REJECTED, CANCELLED).

1.4. Use Cases

1.4.1. Overview Diagram

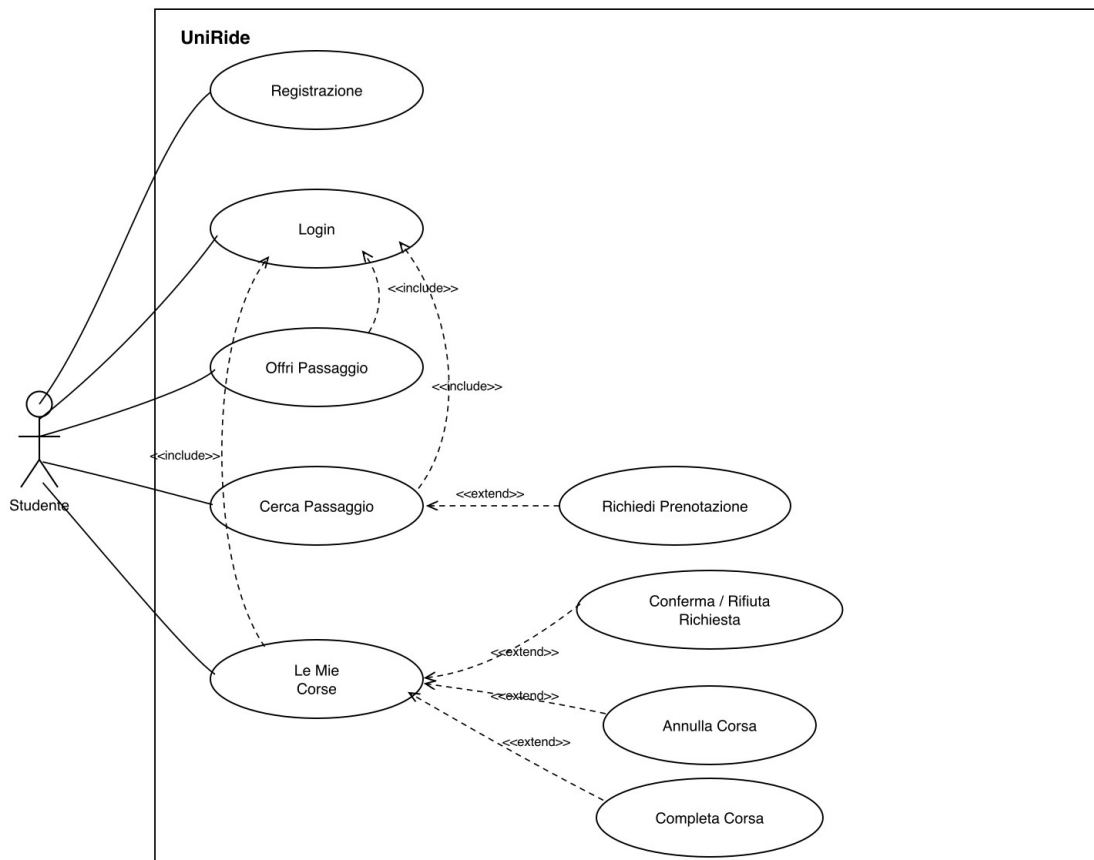


Figura 1 — Diagramma dei casi d'uso, attore Studente

1.4.2. Internal Steps

Nome: Offri un nuovo passaggio

- 1) Il guidatore apre la schermata 'Offri un Passaggio' dalla Dashboard.
- 2) Il guidatore seleziona partenza e destinazione da un menu a tendina di località (editabile per cercare liberamente), una data dal calendario, un orario di partenza (formato HH:mm), i posti disponibili con un contatore numerico e il costo base.
- 3) Il sistema valida i campi obbligatori, verifica che partenza e destinazione non coincidano, che il numero di posti sia maggiore di zero e che l'orario di partenza rispetti il formato HH:mm.
- 4) Il sistema crea l'entità Ride generando un UUID interno e impostando lo stato iniziale AVAILABLE, quindi la persiste tramite il RideDAO ottenuto dalla DAOFactory.
- 5) Il sistema mostra un messaggio di conferma e svuota il form.

Estensioni:

- 3a. Un campo obbligatorio è vuoto, oppure partenza e destinazione coincidono: il sistema mostra un errore inline e resta sulla stessa schermata senza persistere nulla.

- 3b. Il numero di posti non è maggiore di zero: il sistema rifiuta l'input con il messaggio 'Il numero di posti deve essere maggiore di zero.'
- 3c. Il costo inserito è negativo: il sistema rifiuta l'input con il messaggio 'Il costo base non può essere negativo.'
- 3d. L'orario di partenza è assente o non rispetta il formato HH:mm: il sistema rifiuta l'input con il messaggio 'Inserisci un orario di partenza valido (formato HH:mm, es. 08:30).'

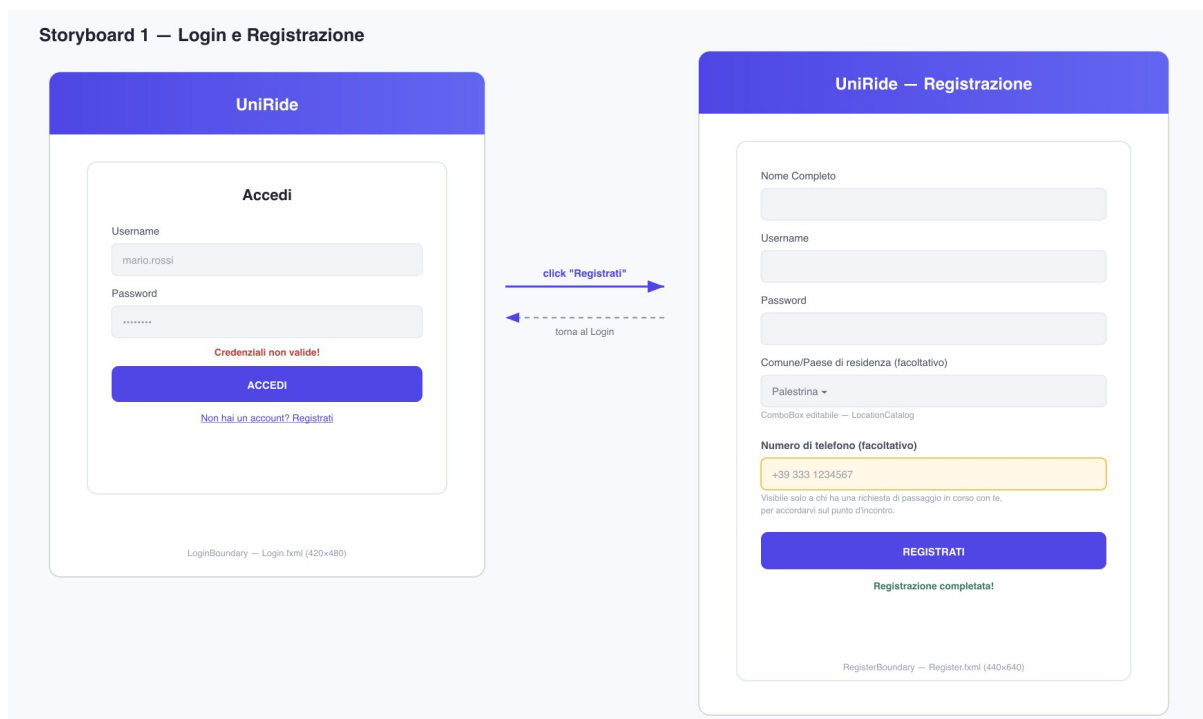
Nome: Richiedi una prenotazione

- 1) Il passeggero cerca i passaggi disponibili per una tratta e ne seleziona uno dai risultati.
- 2) Il sistema verifica che il richiedente non sia il guidatore del passaggio e che non abbia già una richiesta attiva sulla stessa corsa.
- 3) Il sistema riserva immediatamente il posto sul Ride (Pattern State) per evitare overbooking durante l'attesa.
- 4) Il sistema crea un Booking in stato REQUESTED e lo persiste tramite il BookingDAO.
- 5) Il guidatore, dalla propria dashboard, vede la richiesta pendente e la conferma o la rifiuta: se rifiutata, il posto riservato viene liberato.

2. STORYBOARDS

Il primo Storyboard è la schermata di autenticazione, dove lo studente accede con le proprie credenziali o naviga verso il form di registrazione per creare un nuovo account.

Storyboard 1 — Login e Registrazione



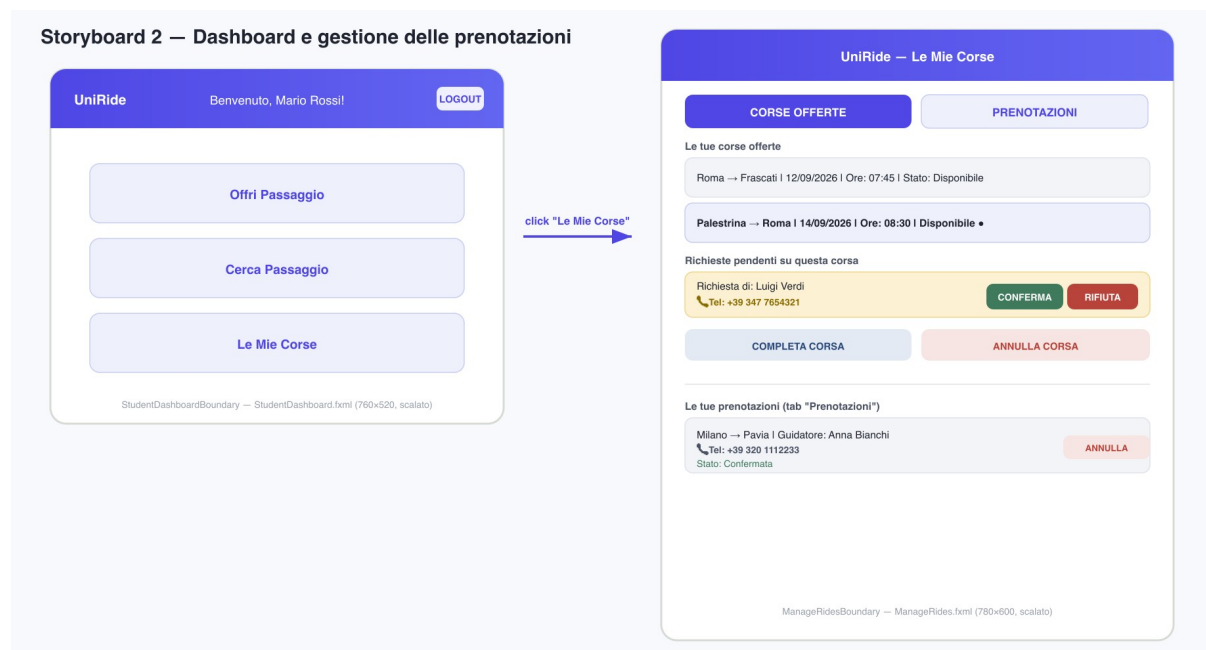
Mockup 1 — Storyboard Login e Registrazione (LoginBoundary / RegisterBoundary)

La schermata di login utilizza il design system moderno introdotto nel progetto: sfondo chiaro con card centrale arrotondata e ombreggiata, palette indaco/violetto (#4f46e5 → #6366f1) coerente su

tutta l'applicazione. Due campi raccolgono username e password. Un'etichetta di errore, inizialmente vuota, appare in rosso sotto i campi quando le credenziali vengono rifiutate dal LoginController. Un bottone secondario porta al form di registrazione, dove nome e cognome, username, password e — facoltativamente — il comune/paese di residenza (selezionabile da un menu a tendina di località) vengono raccolti e validati prima che un'entità Student venga persistita.

Il secondo Storyboard è la Dashboard principale, da cui sono raggiungibili tutte le funzionalità del sistema.

Storyboard 2 — Dashboard e gestione delle prenotazioni



Mockup 2 — Storyboard Dashboard e Le Mie Corse (StudentDashboardBoundary / ManageRidesBoundary)

La Dashboard è costruita su un BorderPane JavaFX. La barra superiore, con sfondo a gradiente indaco/violetto, mostra il logo dell'applicazione, un'etichetta di benvenuto personalizzata col nome dello studente loggato e un bottone Logout allineato a destra. Il centro del layout presenta tre grandi bottoni di navigazione: Offri Passaggio, Cerca Passaggio e Le Mie Corse, che caricano ciascuno la propria vista FXML sostituendo la Scene corrente sullo Stage. La schermata 'Le Mie Corse' è organizzata in due tab: la prima elenca le corse offerte come guidatore, con le richieste di prenotazione pendenti e i bottoni Conferma/Rifiuta, Completa Corsa e Annulla Corsa; la seconda elenca le prenotazioni attive come passeggero, con il relativo stato (In attesa/Confermata) e un bottone per annullarle.

3. DESIGN

In questa sezione vengono espone le principali caratteristiche interne del sistema, con alcuni diagrammi per rappresentarle.

3.1. Class Diagram — VOPC (Analysis)

Data la dimensione del sistema, il diagramma delle classi a livello di analisi viene presentato in un unico diagramma combinato, con le classi raggruppate nelle quattro fasce colorate del pattern Boundary-Control-Entity, più lo strato Bean per i DTO che attraversano i confini architetturali. Le famiglie DAO, troppo numerose per restare leggibili nello stesso diagramma, sono illustrate

separatamente subito dopo; una panoramica globale di come tutti i livelli interagiscono chiude la sezione.

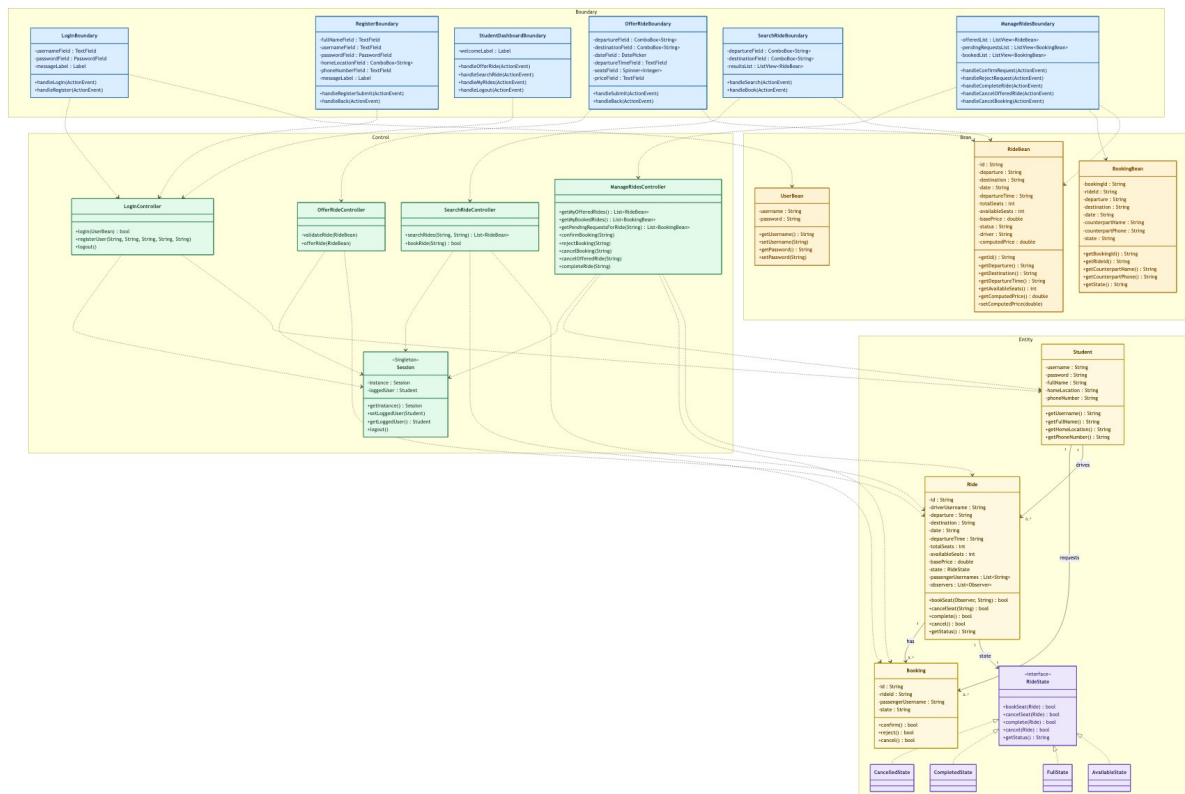


Figura 2 — Diagramma delle classi combinato: Boundary, Bean, Control ed Entity

Lo strato Boundary contiene esclusivamente classi che interagiscono con l'utente: i controller JavaFX collegati alle rispettive viste FXML scambiano solo oggetti Bean con lo strato Control, mai le Entity direttamente. Ogni Boundary gestisce da sé il caricamento del proprio FXML e lo scambio della Scene sullo Stage: non esiste una classe SceneManager centralizzata (CliMain e i vari *CLI offrono un'interfaccia testuale equivalente per ambienti headless, omissi qui per non appesantire il diagramma). Lo strato Bean racchiude i DTO (RideBean, BookingBean, UserBean) che attraversano il confine Boundary ↔ Control: nessuna Entity lascia mai lo strato di dominio verso la UI. Lo strato Control incapsula tutta la logica di business: ogni controller riceve i Bean dalla Boundary, li valida, orchestra le Entity e delega la persistenza allo strato DAO (illustrato separatamente subito dopo); nessuna di queste classi dipende da JavaFX, il che consente agli stessi controller di servire sia la GUI sia la CLI senza modifiche. Il Singleton Session fornisce a tutti i controller l'identità dello studente attualmente autenticato. Lo strato Entity rappresenta infine le classi di dominio: Ride è l'entità più ricca, con la lista degli observer registrati e i metodi che innescano le proprie transizioni di stato (bookSeat, cancelSeat, complete, cancel); Booking modella lo stato di approvazione di ogni singola richiesta di prenotazione, senza duplicare la logica di riserva del posto già gestita da Ride.

La parte DAO — Abstract Factory:

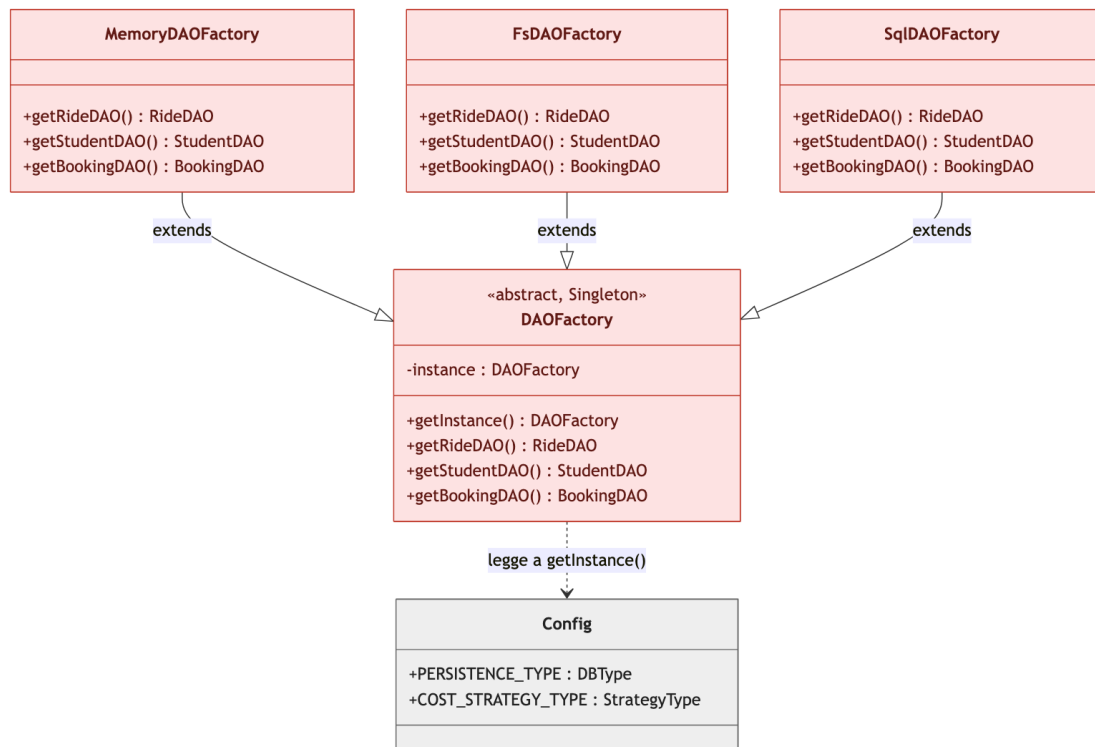


Figura 3 — DAOFactory e le sue Concrete Factory

DAOFactory è l'unico punto d'ingresso allo strato di persistenza. Legge la costante PERSISTENCE_TYPE da Config alla prima invocazione di getInstance() e mette in cache la corrispondente factory concreta, così che ogni controller del sistema riceva oggetti DAO appartenenti alla stessa famiglia coerente.

La parte DAO — famiglia In-Memory:

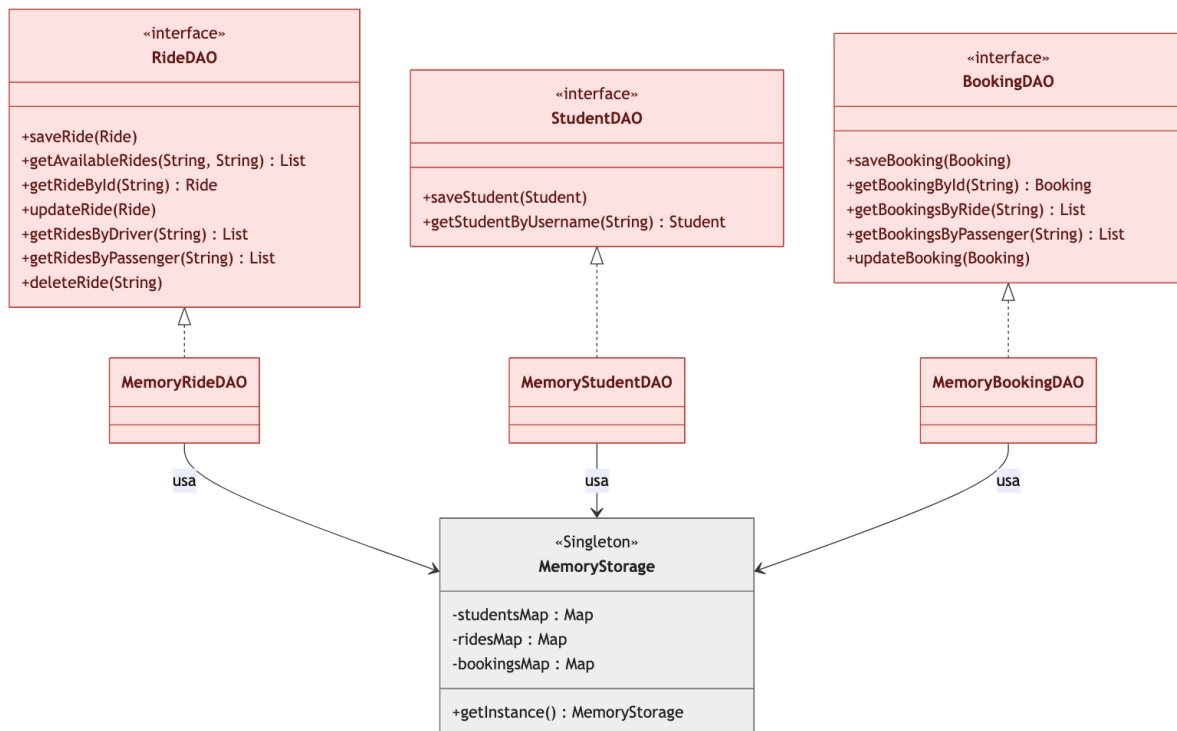


Figura 4 — DAO In-Memory

Le implementazioni In-Memory appoggiano ogni interfaccia su una HashMap statica condivisa tramite il Singleton MemoryStorage, indicizzata per identificativo dell'entità, garantendo un accesso a tempo costante. È la famiglia restituita nella modalità di persistenza di default dell'applicazione.

La parte DAO — famiglia File System:

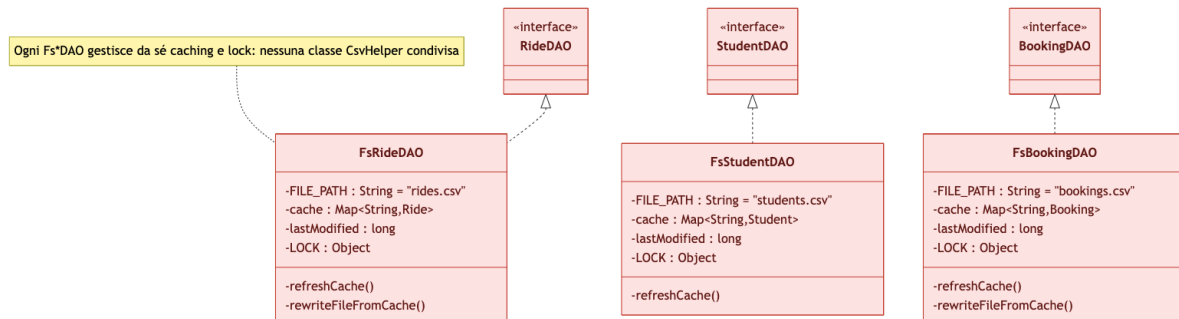


Figura 5 — DAO File System

Le implementazioni File System gestiscono ciascuna in autonomia il proprio file CSV, con un caching in memoria invalidato in base al timestamp di ultima modifica e un lock esplicito per la concorrenza in scrittura: non esiste una classe CsvHelper condivisa fra le tre implementazioni.

La parte DAO — famiglia SQL (database relazionale):

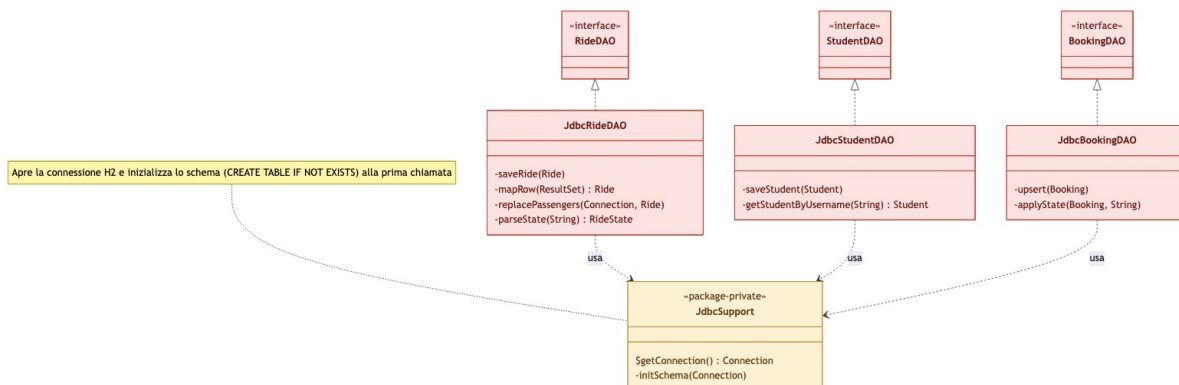


Figura 6 — DAO SQL (H2 via JDBC)

Le implementazioni SQL si appoggiano a JdbcSupport, una classe di supporto package-private che apre la connessione JDBC verso il database H2 embedded e ne inizializza lo schema (CREATE TABLE IF NOT EXISTS) alla prima richiesta. Ogni Jdbc*DAO traduce le proprie operazioni in query preparate: le scritture usano la sintassi di upsert MERGE INTO ... KEY (...) VALUES (...) propria di H2, mentre JdbcRideDAO gestisce in una singola transazione anche la tabella di collegamento ride_passengers, necessaria a rappresentare la relazione molti-a-molti fra un passaggio e i propri passeggeri senza appiattirla in una colonna come nella variante CSV.

La vista globale del diagramma delle classi:

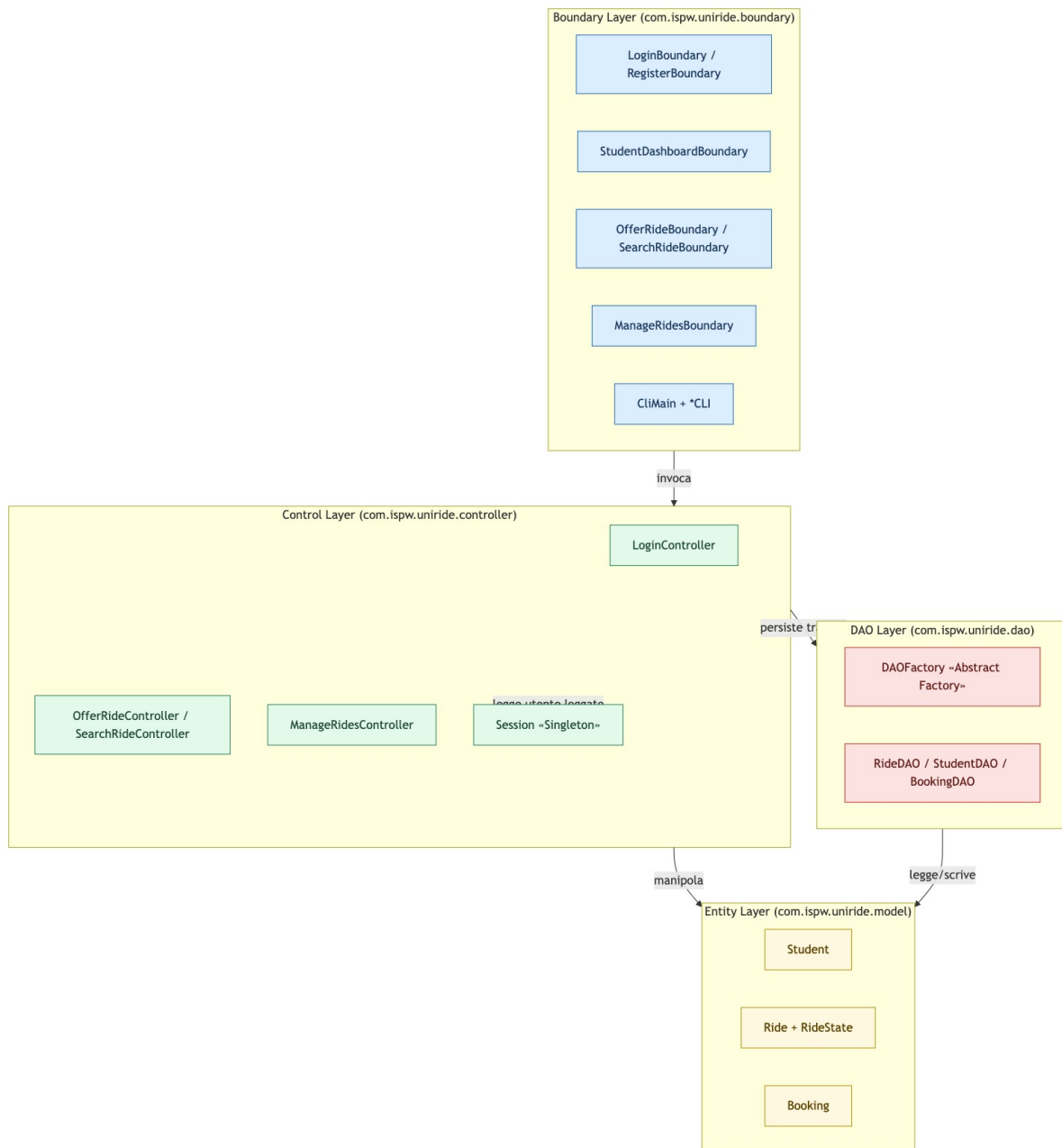


Figura 7 — Panoramica architetturale a livello di package

3.2. Class Diagram — Design Level

Il diagramma a livello di design mette in evidenza i pattern applicati che più elevano il livello ingegneristico del sistema: il Pattern Observer, che disaccoppia la notifica dei passeggeri dal Ride stesso, e il Pattern Strategy, che rende intercambiabile l'algoritmo di suddivisione dei costi.

La parte Pattern:

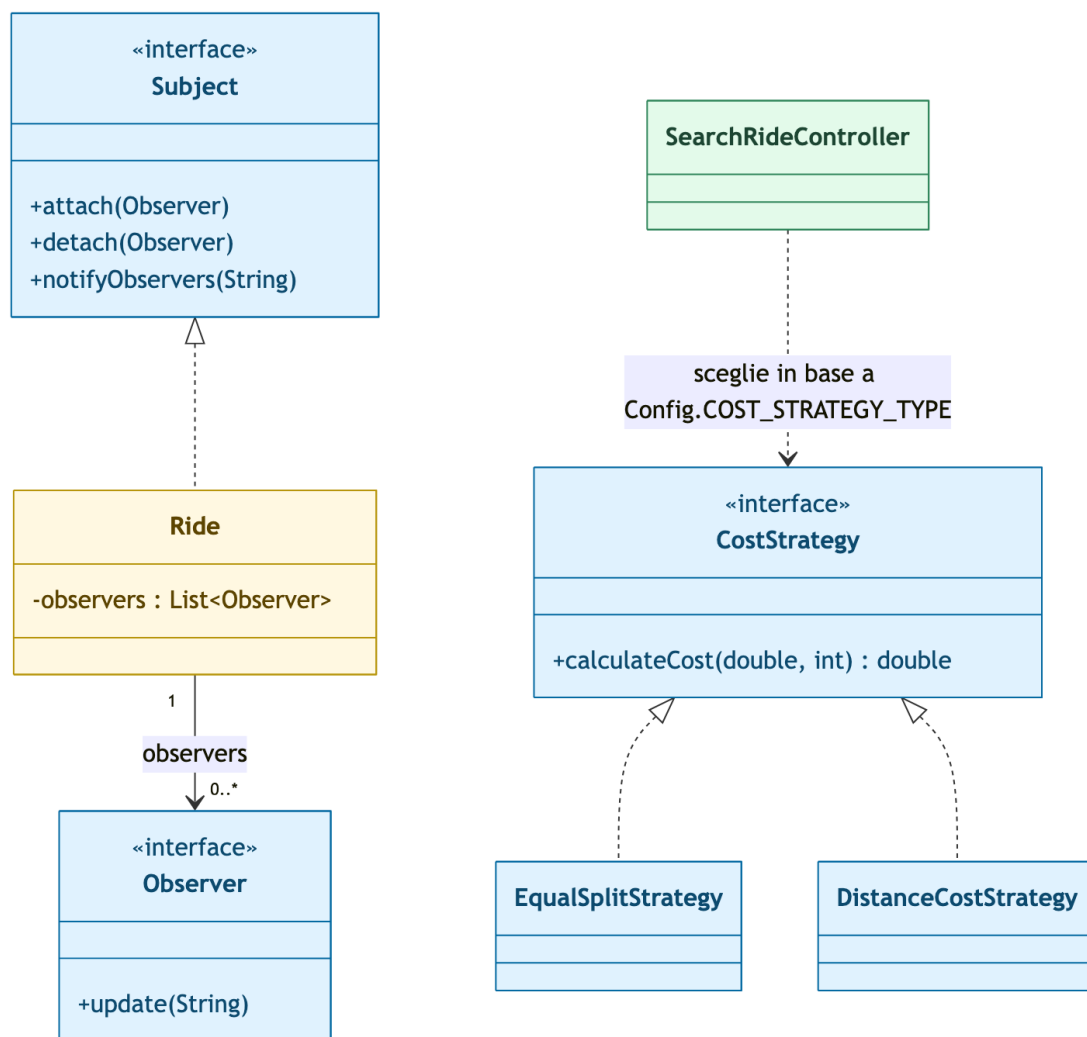


Figura 8 — Observer e Strategy applicati a Ride e al calcolo dei costi

A sinistra, Ride ricopre il ruolo di Subject e mantiene una lista di riferimenti Observer senza conoscerne il tipo concreto (nel codice, un observer è spesso una semplice lambda registrata dal Controller al momento della richiesta). A destra, SearchRideController sceglie dinamicamente quale CostStrategy applicare in base a Config.COST_STRATEGY_TYPE, permettendo di aggiungere un nuovo modello di tariffazione senza toccare il codice esistente.

3.3. Design Patterns

Abstract Factory — DAOFactory

Nel sistema è stato implementato il Pattern Abstract Factory, il cui scopo principale è fornire un'interfaccia per creare famiglie di oggetti correlati senza specificarne le classi concrete. DAOFactory è una classe astratta che espone tre metodi factory: `getRideDAO()`, `getStudentDAO()` e `getBookingDAO()`. Tre sottoclassi concrete la estendono e istanziano le implementazioni corrette in base al valore di Config.PERSISTENCE_TYPE: MemoryDAOFactory (HashMap in RAM), FsDAOFactory (file CSV) e SqlDAOFactory (database relazionale H2 via JDBC). Grazie a questo design, cambiare l'intera applicazione da una tecnologia di persistenza a un'altra richiede di modificare un'unica costante, e nessuna classe Controller è consapevole di quale tecnologia stia effettivamente memorizzando i dati.

Singleton — Session, MemoryStorage e DAOFactory

Il Pattern Singleton garantisce che esista una e una sola istanza di un dato tipo di oggetto, così che tutte le azioni su quell'oggetto siano eseguite in modo coerente. Session mantiene lo Student attualmente autenticato ed è raggiungibile globalmente tramite Session.getInstance(). MemoryStorage centralizza le HashMap condivise dalla famiglia di DAO in memoria. DAOFactory stessa mette in cache la factory concreta scelta in un campo statico protetto da un accessor synchronized.

State — RideState

Il Pattern State modella il ciclo di vita di un passaggio. L'interfaccia RideState definisce i quattro stati concreti che un Ride può assumere (AvailableState, FullState, CompletedState, CancelledState), e ogni transizione è innescata esclusivamente da un metodo su Ride stesso (bookSeat, cancelSeat, complete, cancel), così che il campo state non possa mai divergere dal reale numero di posti disponibili né da transizioni non ammesse (i due stati terminali COMPLETED e CANCELLED rifiutano qualunque ulteriore transizione). L'entità Booking adotta invece un approccio più leggero: uno stato testuale con transizioni validate direttamente dai propri metodi confirm()/reject()/cancel(), senza una gerarchia di classi State dedicata.

Strategy — CostStrategy

Il Pattern Strategy incapsula una famiglia di algoritmi intercambiabili. CostStrategy dichiara il singolo metodo calculateCost(basePrice, numPassengers). EqualSplitStrategy divide il costo totale in parti uguali fra guidatore e passeggeri, mentre DistanceCostStrategy calcola il costo in base alla distanza. SearchRideController sceglie dinamicamente quale strategia istanziare in base a Config.COST_STRATEGY_TYPE ad ogni ricerca, così l'algoritmo può essere cambiato modificando un'unica costante.

Observer — Ride e i passeggeri iscritti

Il Pattern Observer definisce una dipendenza uno-a-molti così che, quando un oggetto cambia stato, tutti i suoi dipendenti vengano notificati automaticamente. Ride mantiene una lista di istanze Observer e invoca notifyObservers() ogni volta che un posto viene richiesto, liberato, o la corsa viene annullata. Nel prototipo, l'Observer registrato alla richiesta di un posto è tipicamente una funzione lambda che registra la notifica tramite LoggerCustom.

3.4. Activity Diagram

Il seguente diagramma di attività descrive il ramo principale del flusso di login/registrazione insieme alle funzionalità principali del sistema:

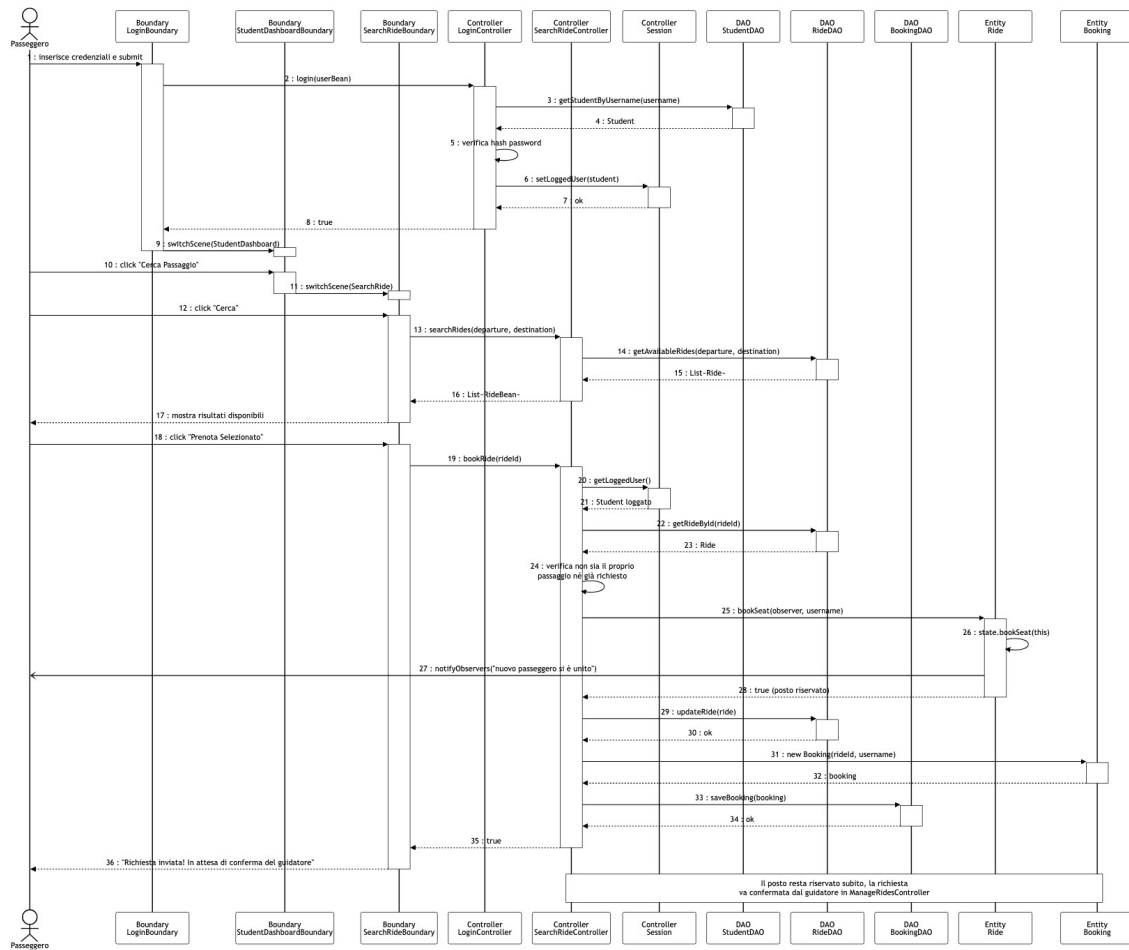


Figura 10 — Sequence Diagram del flusso login → ricerca → richiesta di prenotazione

3.6. State Diagram

Due macchine a stati governano il sistema. La prima descrive il ciclo di vita di un Ride:

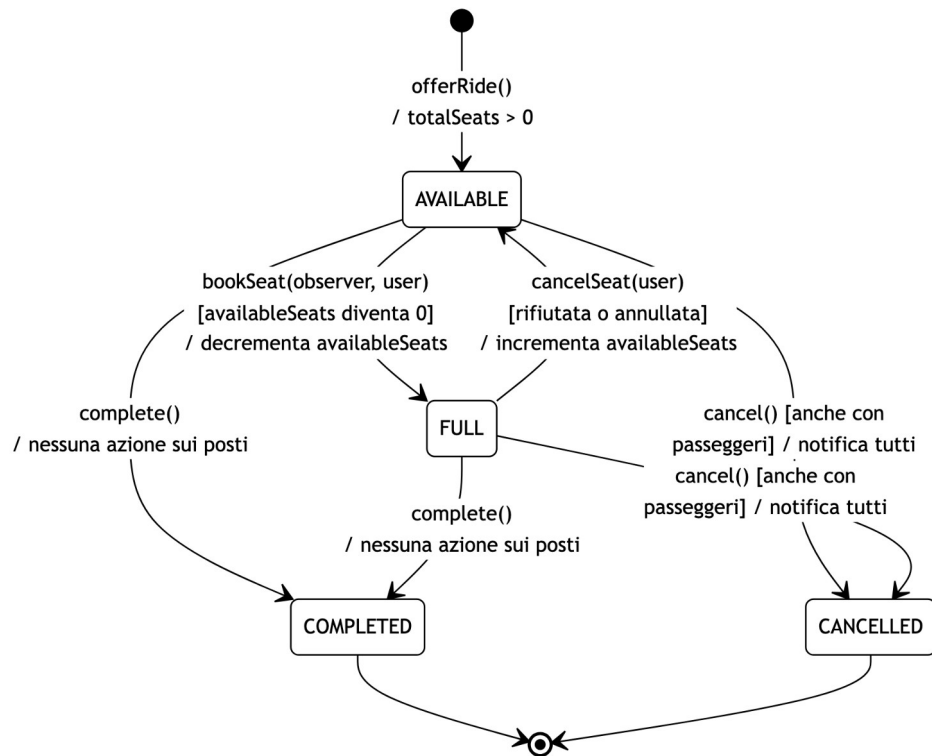


Figura 11 — State Diagram di Ride

La seconda descrive il ciclo di vita di un Booking. Sia il rifiuto sia l'annullamento innescano il rilascio del posto riservato sul Ride associato:

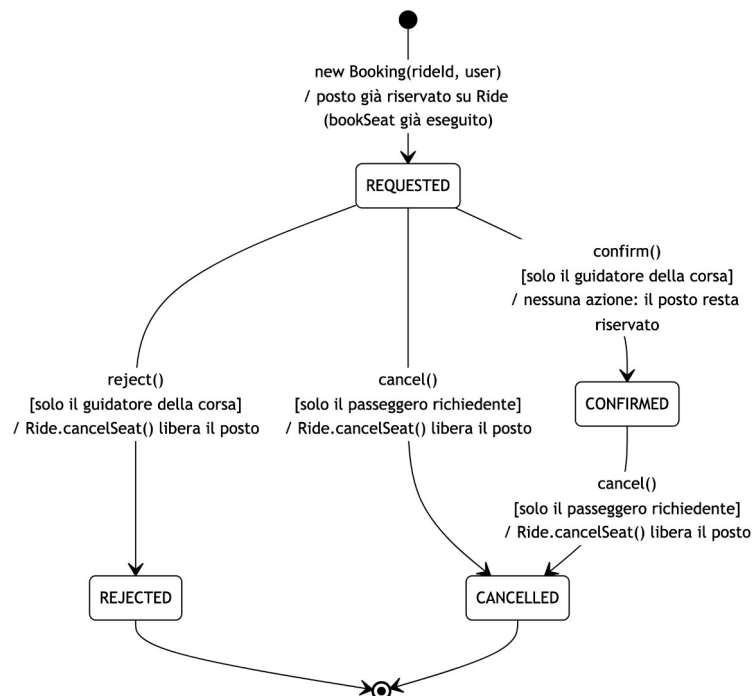


Figura 12 — State Diagram di Booking

4. TESTING

Oltre alle classi che verificano lo strato di dominio e i pattern applicati, la suite include una classe di test dedicata per ciascuna famiglia di DAO (In-Memory, File System e SQL), così da coprire anche lo strato di persistenza e non solo la logica di business.

Classe di Test	Casi di Test
RideStateTest	stato iniziale AVAILABLE; transizione a FULL all'ultimo posto; rifiuto prenotazione a macchina piena; ripristino stato AVAILABLE alla cancellazione
EqualSplitStrategyTest	suddivisione equa del costo con più passeggeri; nessun passeggero (costo interamente a carico del guidatore); divisione 50/50 con un passeggero
BookingWorkflowTest	nuova richiesta in stato REQUESTED; conferma di una richiesta pendente; rifiuto con rilascio del posto; impossibilità di confermare/rifiutare una richiesta già gestita; annullamento da parte del passeggero
RideLifecycleTest	completamento da AVAILABLE e da FULL; uno stato completato è terminale (nessuna nuova prenotazione, nessuna doppia transizione); annullamento anche con passeggeri a bordo e relativa notifica Observer; uno stato annullato è terminale
JdbcRideDAOTest, JdbcStudentDAOTest, JdbcBookingDAOTest	salvataggio e riletture tramite il database H2 reale (upsert via MERGE INTO, ricostruzione dello stato e dei passeggeri di un Ride, applicazione delle transizioni di un Booking); ogni test usa identificativi univoci (System.nanoTime()) per restare isolato nonostante il file H2 sia condiviso fra le esecuzioni
SqlDAOFactoryTest	verifica che SqlDAOFactory restituisca effettivamente le implementazioni Jdbc*DAO e non uno stub

I test sullo strato di dominio istanziano direttamente le Entity (Ride, Booking) senza passare dai DAO o dalla Sessione, così da restare completamente deterministici e indipendenti dalla macchina su cui vengono eseguiti; i test sui Jdbc*DAO usano invece il vero database H2 embedded, a riprova che lo strato SQL non è solo dichiarato ma effettivamente funzionante. Il progetto include inoltre il plugin JaCoCo per la generazione del report di code coverage, letto automaticamente dall'analisi SonarCloud (vedi Deliverable Links a fine documento).

DELIVERABLE LINKS

Come richiesto dalle Final Project Detailed Instructions, i deliverable delle sezioni 5 (Code) e 6 (Video) non sono riportati per intero in questo PDF, ma referenziati tramite i link seguenti.

Codice sorgente (comprende anche i test JUnit)

<https://github.com/AlessioCarta/ISPW>

Analisi statica del codice (SonarCloud — 0 bug, 0 vulnerabilità, 0 code smell)

https://sonarcloud.io/project/overview?id=AlessioCarta_ISPW

Video dimostrativo (1-2 minuti, con audio)

<https://github.com/AlessioCarta/Video-UniRide>